

PRACTICA CALIFICADA 2

1. PREGUNTA 1

1.1. (3 puntos) Indicar los datos de salida y entrada del problema y de su operación, así como, cualquier otro(s) requisito(s) de entrada o salida que considere conveniente.

1.1.1. Enunciado simulado del problema

El problema se desarrolla en el contexto de una biblioteca que administra libros mediante una lista enlazada simple. Cada libro será almacenado como un objeto de tipo Libro; por lo tanto, el elemento principal de cada nodo no será un String, ni un tipo primitivo, sino un TAD definido por el programador.

Cada nodo de la lista enlazada contendrá dos partes: el dato, que será un objeto Libro, y la referencia al siguiente nodo de la lista.

La lista inicial tendrá como mínimo tres libros registrados. Sobre esta lista se aplicará la operación asignada número 7, la cual consiste en retornar la posición donde se encuentra por primera vez un valor dentro de la lista enlazada.

En este caso, el valor a buscar será el ISBN del libro, debido a que permite identificar de manera única a cada libro dentro de la biblioteca.

1.1.2. Operación asignada

La operación asignada es la siguiente: Operación 7: Retornar la posición donde encontró por primera vez un valor de un elemento de la lista enlazada utilizando como parámetro el valor a buscar. Para este problema, la operación se aplicará buscando el ISBN de un libro dentro de una lista enlazada simple de libros.

El método correspondiente será: `buscarPrimeraPosicion(String isbnBuscado)`. Este método recibirá como parámetro el ISBN que se desea buscar y retornará la posición donde se encuentre por primera vez dentro de la lista enlazada. Si el ISBN no existe en la lista, el método retornará -1.

1.1.3. TAD utilizado

El TAD utilizado será la clase Libro

Cada libro tendrá los siguientes atributos:

Dato	Tipo de dato	Descripción
codigo	String	Código interno del libro dentro de la biblioteca.
isbn	String	Identificador único del libro. Se usará como criterio de búsqueda.
titulo	String	Título del libro.
anioPublicacion	int	Año de publicación del libro.

Aunque algunos atributos internos son de tipo String o int, el elemento almacenado en la lista enlazada no es un dato primitivo, sino un objeto completo de tipo Libro.

1.1.4. Datos de entrada del problema

Los datos de entrada del problema son los libros que se cargarán inicialmente en la lista enlazada.

Código	ISBN	Título	Año
L001	978001	Estructuras de Datos	2021
L002	978002	Programación en Java	2020
L003	978003	Algoritmos Aplicados	2022

Estos tres libros permiten cumplir con el requisito de trabajar con una lista enlazada de mínimo tres elementos TAD.

1.1.5. Datos de entrada de la operación

Para la operación asignada número 7, el dato de entrada será el valor a buscar dentro de la lista enlazada.

En este caso, el valor a buscar será el ISBN del libro.

Entrada	Tipo de dato	Descripción
isbnBuscado	String	ISBN del libro que se desea buscar dentro de la lista enlazada.

Ejemplo de entrada para la operación:

String isbnBuscado = "978002"; Este valor corresponde al ISBN del libro Programación en Java, el cual se encuentra en la posición 2 de la lista enlazada inicial.

1.1.6. Datos de salida del problema

Como salida general del problema se mostrará el estado inicial de la lista enlazada y el resultado de la operación asignada.

Salida	Descripción
Lista inicial	Muestra los libros cargados antes de aplicar la operación.
Resultado de la operación	Muestra la primera posición donde fue encontrado el ISBN buscado.
Mensaje de no encontrado	Se muestra si el ISBN no existe dentro de la lista enlazada.

1.1.7. Datos de salida de la operación

La operación buscarPrimeraPosicion(String isbnBuscado) retornará un valor entero.

Resultado	Significado
1, 2, 3, ...	Posición donde se encontró por primera vez el libro buscado.
-1	El libro no fue encontrado en la lista enlazada.

Ejemplo de salida esperada:

ISBN buscado: 978002

Primera posición encontrada: 2

Si el ISBN no existe:

ISBN buscado: 999999

Libro no encontrado.

1.1.8. Requisitos de entrada

Para que el algoritmo funcione correctamente, se consideran los siguientes requisitos:

- La lista enlazada debe tener como mínimo tres libros cargados inicialmente.
- Cada elemento de la lista debe ser un objeto de tipo Libro.
- No se debe almacenar directamente un String, int, double, boolean u otro tipo primitivo como elemento principal de la lista.
- El TAD utilizado no debe pertenecer al proyecto desarrollado en la pregunta 2.
- El valor a buscar será el ISBN del libro.
- El ISBN buscado no debe ser nulo ni estar vacío.
- La búsqueda debe iniciar desde el primer nodo de la lista.
- Las posiciones de la lista se manejarán desde 1.
- Si la lista está vacía, el método debe retornar -1.
- Si el ISBN no se encuentra en la lista, el método debe retornar -1.

1.1.9. Requisitos de salida

La salida debe permitir verificar claramente el resultado de la operación.

Se considera conveniente mostrar:

- La lista enlazada inicial.
- El ISBN que será buscado.
- La posición donde se encontró por primera vez el libro.
- Un mensaje indicando que el libro no fue encontrado, si el método retorna -1.

1.1.10. Resumen técnico de la operación

La operación número 7 realiza una búsqueda secuencial dentro de una lista enlazada simple. Para ello, se utiliza una variable auxiliar llamada actual, que comienza apuntando al nodo inicial de la lista. También se utiliza una variable posicion, que inicia en 1 y aumenta cada vez que se avanza al siguiente nodo. En cada nodo se compara el ISBN del libro almacenado con el ISBN buscado. Si ambos valores son iguales, el método retorna la posición actual. Si el recorrido llega al final de la lista sin encontrar coincidencias, el método retorna -1.

1.2. (4 puntos) Programar el algoritmo en Java que incluya el método y ejecutar la operación con el TAD. Incluir comentarios de código que considere importante. Adjuntar el archivo de proyecto empaquetado.

1.2.1. Programa en Java

```
2. package edu.pe.utp.pc2.ejercicio1.modelo;

/**
 * TAD Libro.
 * Esta clase representa el dato que será almacenado
 * dentro de cada nodo de la lista enlazada.
 * Importante:
 * - El elemento principal de la lista NO es String ni tipo primitivo.
 * - Cada nodo almacena un objeto completo de tipo Libro.
 */
public class Libro {

    private String codigo;
    private String isbn;
    private String titulo;
    private int anioPublicacion;

    /**
     * Constructor principal del TAD Libro.
     */
    public Libro(String codigo, String isbn, String titulo, int
anioPublicacion) {
        this.codigo = codigo;
        this.isbn = isbn;
        this.titulo = titulo;
        this.anioPublicacion = anioPublicacion;
    }

    public String getCodigo() {
        return codigo;
    }

    public String getIsbn() {
        return isbn;
    }
}
```

```
        public String getTitulo() {
            return titulo;
        }

        public int getAnioPublicacion() {
            return anioPublicacion;
        }

        /**
         * Permite mostrar el contenido del libro de forma clara
         * cuando se imprime la lista enlazada.
         */
        @Override
        public String toString() {
            return "Libro{" + "codigo='" + codigo + '\'' + ", isbn='" +
            isbn + '\'' + ", titulo='" + titulo + '\'' + ", anioPublicacion='" +
            anioPublicacion + '\'';
        }
    }
}
```

```
package edu.pe.utp.pc2.ejercicio1.estructura;

import edu.pe.utp.pc2.ejercicio1.modelo.Libro;

/**
 * NodoLibro representa un nodo de una lista enlazada simple.
 * Cada nodo contiene:
 * - Un dato de tipo Libro.
 * - Una referencia al siguiente nodo.
 */
public class NodoLibro {

    private Libro dato;
    private NodoLibro siguiente;

    /**
     * Al crear un nodo, recibe un objeto Libro.
     * no se encuentra enlazado con otro nodo.
     */
    public NodoLibro(Libro dato) {
        this.dato = dato;
        this.siguiente = null;
    }

    public Libro getDato() {
        return dato;
    }

    public void setDato(Libro dato) {
        this.dato = dato;
    }

    public NodoLibro getSiguiente() {
        return siguiente;
    }
}
```

```
public void setSiguiente(NodoLibro siguiente) {  
    this.siguiente = siguiente;  
}  
}
```

```
package edu.pe.utp.pc2.ejercicio1.estructura;  
  
import edu.pe.utp.pc2.ejercicio1.modelo.Libro;  
  
/**  
 * Lista enlazada simple para almacenar objetos Libro.  
 * La variable inicio guarda la referencia del primer nodo.  
 */  
public class ListaLibros {  
  
    private NodoLibro inicio;  
  
    public ListaLibros() {  
        this.inicio = null;  
    }  
  
    /**  
     * Inserta un libro al final de la lista enlazada.  
     * Este metodo permite cargar los tres elementos minimos del enunciado.  
     */  
    public void insertarAlFinal(Libro libro) {  
        NodoLibro nuevoNodo = new NodoLibro(libro);  
  
        if (inicio == null) {  
            inicio = nuevoNodo;  
            return;  
        }  
  
        NodoLibro actual = inicio;  
  
        while (actual.getSiguiente() != null) {  
            actual = actual.getSiguiente();  
        }  
  
        actual.setSiguiente(nuevoNodo);  
    }  
  
    /**  
     * Operacion 7:  
     * Retorna la posicion donde se encontro por primera vez un valor.  
     * En este caso, el valor a buscar es el ISBN del libro.  
     *  
     * @param isbnBuscado ISBN del libro que se desea ubicar.  
     * @return posicion donde se encontro por primera vez; -1 si no existe.  
     */  
    public int buscarPrimeraPosicion(String isbnBuscado) {
```

```
        if (isbnBuscado == null || isbnBuscado.trim().isEmpty()) {
            return -1;
        }

        String isbnNormalizado = isbnBuscado.trim();
        NodoLibro actual = inicio;
        int posicion = 1;

        while (actual != null) {
            if (actual.getDato().getIsbn().equalsIgnoreCase(isbnNormalizado))
            {
                return posicion;
            }

            // Avanza la referencia actual y actualiza la posicion logica.
            actual = actual.getSiguiente();
            posicion++;
        }

        return -1;
    }

    /**
     * Muestra todos los libros almacenados en la lista enlazada.
     */
    public void mostrarLista() {
        if (inicio == null) {
            System.out.println("La lista esta vacia.");
            return;
        }

        NodoLibro actual = inicio;
        int posicion = 1;

        while (actual != null) {
            System.out.println("Posicion " + posicion + ": " +
actual.getDato());
            actual = actual.getSiguiente();
            posicion++;
        }
    }
}
```

```
package edu.pe.utp.pc2.ejercicio1.app;

import edu.pe.utp.pc2.ejercicio1.estructura.ListaLibros;
import edu.pe.utp.pc2.ejercicio1.modelo.Libro;

/**
 * Clase principal del programa.
 * Aqui se crea una lista enlazada simple de libros y se ejecuta la operacion
7.
 */
public class Main {
```



```
public static void main(String[] args) {

    ListaLibros lista = new ListaLibros();

    // Cada Libro es un TAD; la lista no almacena String ni tipos
    primitivos.
    Libro libro1 = new Libro("L001", "978001", "Estructuras de Datos",
2021);
    Libro libro2 = new Libro("L002", "978002", "Programacion en Java",
2020);
    Libro libro3 = new Libro("L003", "978003", "Algoritmos Aplicados",
2022);

    // Se cargan tres elementos como minimo, segun pide el enunciado.
    lista.insertarAlFinal(libro1);
    lista.insertarAlFinal(libro2);
    lista.insertarAlFinal(libro3);

    System.out.println("==== LISTA INICIAL DE LIBROS ====");
    lista.mostrarLista();

    // Operacion 7: retornar la primera posicion donde aparece un valor.
    String isbnBuscado = "978002";

    System.out.println("\n==== OPERACION 7: BUSCAR PRIMERA POSICION
====");
    System.out.println("ISBN buscado: " + isbnBuscado);

    int posicionEncontrada = lista.buscarPrimeraPosicion(isbnBuscado);

    if (posicionEncontrada != -1) {
        System.out.println("Primera posicion encontrada: " +
posicionEncontrada);
    } else {
        System.out.println("Libro no encontrado.");
    }

    // Caso adicional para demostrar el retorno -1 cuando el ISBN no
    existe.
    String isbnNoExistente = "9999999";
    int posicionNoEncontrada =
lista.buscarPrimeraPosicion(isbnNoExistente);
    System.out.println("\nISBN no existente: " + isbnNoExistente);
    System.out.println("Resultado de busqueda fallida: " +
posicionNoEncontrada);
}
}
```

2.1.1. Impresión en consola

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.2\lib\idea_rt.
==== LISTA INICIAL DE LIBROS ====
Posicion 1: Libro{codigo='L001', isbn='978001', titulo='Estructuras de Datos', anioPublicacion=2021}
Posicion 2: Libro{codigo='L002', isbn='978002', titulo='Programacion en Java', anioPublicacion=2020}
Posicion 3: Libro{codigo='L003', isbn='978003', titulo='Algoritmos Aplicados', anioPublicacion=2022}

==== OPERACION 7: BUSCAR PRIMERA POSICION ====
ISBN buscado: 978002
Primera posicion encontrada: 2

ISBN no existente: 999999
Resultado de busqueda fallida: -1

Process finished with exit code 0
```

2.2. (3 puntos) Explicar el cambio de memoria de la operación seleccionada (no la de recorrido). La explicación puede realizar un esquema en Excel, usar el depurador del Netbeans u otro recurso que considere conveniente. No puede utilizar librería.

La operación seleccionada es la **operación 7**, la cual consiste en retornar la posición donde se encuentra por primera vez un valor dentro de una lista enlazada.

En este caso, el valor a buscar será el ISBN de un libro. Por ello, el método implementado es:

```
buscarPrimeraPosicion(String isbnBuscado)
```

La operación se ejecuta sobre una lista enlazada simple de libros. Cada nodo contiene un objeto de tipo `Libro` y una referencia al siguiente nodo.

Código central de la operación

```
public int buscarPrimeraPosicion(String isbnBuscado) {

    if (isbnBuscado == null || isbnBuscado.trim().isEmpty()) {
        return -1;
    }

    String isbnNormalizado = isbnBuscado.trim();
    NodoLibro actual = inicio;
    int posicion = 1;

    while (actual != null) {

        if (actual.getDato().getIsbn().equalsIgnoreCase(isbnNormalizado)) {
            return posicion;
        }
    }
}
```

```
        actual = actual.getSiguiente();  
        posicion++;  
    }  
  
    return -1;  
}
```

Estado inicial de la lista enlazada

Para la ejecución se cargan tres libros en la lista enlazada:

inicio -> Nodo(L001) -> Nodo(L002) -> Nodo(L003) -> null

Representación detallada:

```
inicio  
↓  
[Nodo 1]  
dato = Libro{codigo='L001', isbn='978001', titulo='Estructuras de Datos'}  
siguiente —————  
↓  
[Nodo 2]  
dato = Libro{codigo='L002', isbn='978002',  
titulo='Programación en Java'}  
siguiente —————  
↓  
[Nodo 3]  
dato =  
Libro{codigo='L003', isbn='978003', titulo='Algoritmos Aplicados'}  
siguiente = null
```

El valor buscado será:

```
String isbnBuscado = "978002";
```

Este ISBN corresponde al segundo libro de la lista, por lo tanto el resultado esperado de la operación es:

Primera posición encontrada: 2

Variables de memoria utilizadas en la operación

Variable	Tipo	Función dentro del método
isbnBuscado	String	Recibe el ISBN enviado como parámetro de búsqueda.
isbnNormalizado	String	Guarda el ISBN sin espacios al inicio o al final.
inicio	NodoLibro	Referencia al primer nodo de la lista enlazada.
actual	NodoLibro	Referencia auxiliar que apunta al nodo que se está evaluando.
posicion	int	Guarda la posición lógica del nodo actual dentro de la lista.
return	int	Devuelve la posición encontrada o -1 si no existe coincidencia.

Tabla de cambio de estado de memoria

Paso	Instrucción ejecutada	isbnBuscado	isbnNormalizado	actual apunta a	posicion	Comparación realizada	Resultado
1	Se invoca el método buscarPrimeraPosicion("978002")	"978002"	Sin asignar	Sin asignar	Sin asignar	Aún no compara	Ingresa el parámetro al método
2	if (isbnBuscado == null isbnBuscado.trim().isEmpty())	"978002"	Sin asignar	Sin asignar	Sin asignar	El ISBN no es nulo ni vacío	Continúa la ejecución
3	String isbnNormalizado = isbnBuscado.trim();	"978002"	"978002"	Sin asignar	Sin asignar	No compara nodos todavía	Se normaliza el ISBN buscado
4	NodoLibro actual = inicio;	"978002"	"978002"	Nodo(L001)	Sin asignar	No compara todavía	actual apunta al primer nodo
5	int posicion = 1;	"978002"	"978002"	Nodo(L001)	1	No compara todavía	Se inicia la posición en 1
6	while (actual != null)	"978002"	"978002"	Nodo(L001)	1	actual no es null	Ingresa al ciclo

Paso	Instrucción ejecutada	isbnBuscado	isbnNormalizado	actual apunta a	posicion	Comparación realizada	Resultado
7	actual.getDato().getIsbn().equalsIgnoreCase(isbnNormalizado)	"978002"	"978002"	Nodo(L001)	1	978001 vs 978002	No coincide
8	actual = actual.getSiguiente();	"978002"	"978002"	Nodo(L002)	1	No compara en esta instrucción	actual avanza al segundo nodo
9	posicion++;	"978002"	"978002"	Nodo(L002)	2	No compara en esta instrucción	La posición aumenta a 2
10	while (actual != null)	"978002"	"978002"	Nodo(L002)	2	actual no es null	Ingresa nuevamente al ciclo
11	actual.getDato().getIsbn().equalsIgnoreCase(isbnNormalizado)	"978002"	"978002"	Nodo(L002)	2	978002 vs 978002	Coincide
12	return posicion;	"978002"	"978002"	Nodo(L002)	2	Coincidencia encontrada	Retorna 2 y finaliza el método

Explicación técnica del cambio de memoria

Al iniciar el método, el parámetro `isbnBuscado` recibe el valor "978002". Luego se valida que dicho valor no sea `null` ni esté vacío. Como el valor es válido, se crea la variable `isbnNormalizado`, que almacena el ISBN sin espacios adicionales.

Después, la variable `actual` recibe la referencia almacenada en `inicio`. Esto significa que `actual` apunta inicialmente al primer nodo de la lista, es decir, al nodo que contiene el libro con código L001.

La variable `posicion` inicia con el valor 1, porque las posiciones de la lista se están manejando desde uno. En la primera iteración, el método compara el ISBN del libro ubicado en el primer nodo, 978001, con el ISBN buscado, 978002. Como no coinciden, la referencia `actual` avanza al siguiente nodo mediante la instrucción:

```
actual = actual.getSiguiente();
```

Luego, la variable `posicion` aumenta de 1 a 2 mediante la instrucción:

```
posicion++;
```

En la segunda iteración, `actual` apunta al segundo nodo de la lista, que contiene el libro con ISBN 978002. En este punto, la comparación sí coincide, por lo que el método ejecuta:

```
return posicion;
```

Como `posicion` tiene el valor 2, el método retorna 2 y finaliza inmediatamente la búsqueda.

Estado final de la lista enlazada

La lista enlazada no sufre cambios estructurales, porque la operación 7 es una operación de búsqueda y no una operación de inserción, eliminación o modificación.

Antes de ejecutar la operación:

```
inicio -> Nodo(L001) -> Nodo(L002) -> Nodo(L003) -> null
```

Después de ejecutar la operación:

```
inicio -> Nodo(L001) -> Nodo(L002) -> Nodo(L003) -> null
```

Por lo tanto, los enlaces entre nodos se mantienen iguales:

```
Nodo(L001).siguiente -> Nodo(L002)
```

```
Nodo(L002).siguiente -> Nodo(L003)  
Nodo(L003).siguiente -> null
```

Conclusión del cambio de estado de memoria

Durante la operación 7 no se crea ningún nodo nuevo, no se elimina ningún nodo y tampoco se modifica el contenido de los libros almacenados en la lista. El cambio de estado ocurre únicamente en las variables locales del método.

La variable `actual` cambia de referencia conforme avanza por los nodos de la lista, y la variable `posicion` aumenta en una unidad por cada nodo visitado. Cuando el ISBN del nodo actual coincide con el ISBN buscado, el método retorna la posición correspondiente.

En la ejecución realizada, el ISBN buscado fue 978002. Este valor se encontró en el segundo nodo de la lista enlazada, por lo que el método retornó la posición 2.